MARLET: Multi-Agent Retrieval with Lightweight Evidence and Task-State Control

Anonymous Authors

Abstract— Large language models can generate fluent responses, but reliable domain assistance requires grounding in external evidence, visible user or task state, and explicit answer criteria. Classical retrieval-augmented generation (RAG) usually treats grounding as a fixed retrieve-then-read step in which a query is matched against a corpus and the generator reads the returned passages. This control flow is rigid when the same surface question requires different evidence, response structure, or evaluation criteria across situations. We propose MARLET (Multi-Agent Retrieval with Lightweight Evidence and Task-State Control), a configurable multi-agent retrieval framework that treats RAG as structured prompt construction rather than query-only passage lookup. A supervisory router identifies the response regime; LLM-backed planning, state diagnosis, criteria analysis, and conditional retrieval prepare typed context blocks; and all intermediate outputs are reassembled into one augmented prompt for a fixed generator and bounded critic. The reusable mechanism is domain-general: only cue lexicons, default criteria, and corpus/index contents are domain configuration. Controlled validation in educational tutoring under matched backbone, corpus, reranker, and response cap shows that MARLET improves quality proxies while reducing token use, with additional latency and model-call reductions in some settings.

I. Introduction

Large language models (LLMs) are increasingly used as domain assistants because they can generate explanations, diagnose errors, and provide feedback. However, a useful assistant cannot rely only on parametric knowledge. Knowledge-grounded dialogue has long shown the value of external evidence [1]; domain assistance similarly needs grounding in manuals, policies, evaluation criteria, worked examples, user profiles, or task records to avoid fluent but misaligned guidance. Educational tutoring is a demanding instantiation of this broader problem [2], [3].

Classical retrieval-augmented generation (RAG) has therefore become the default grounding mechanism [4]. In classical RAG, the query is embedded, the top-k most similar passages are retrieved, and the LLM generates a response conditioned on those passages. Thus, relevance depends heavily on retrieval quality and query representation [5].

Domain assistance is more complex than factual question answering. The appropriate response may depend on expertise, goal, task history, constraints, prior confusion, and evaluation criteria. We call these visible attributes the user's personalized task state. Retrieval must therefore find evidence appropriate for the task state, not merely topical evidence.

Classical RAG can include explicit state text when supplied, but appending user state, dialogue, criteria, and task constraints forces one query embedding to compress topic, level of detail, intent, and response requirements. As constraints grow, search intent can blur. The issue is therefore how a response system should organize constraints before retrieval and generation.

Therefore, we introduce MARLET (Multi-Agent Retrieval with Lightweight Evidence and Task-State Control), a configurable RAG controller. Instead of letting the retriever act as the top-level controller, a supervisory router chooses the response regime and retrieval gate; planner, state, and criteria specialists prepare typed context in parallel; bounded local tools inspect resources; and all outputs are reassembled with evidence into one generator prompt.

The experiments validate the framework rather than present a model leaderboard. We compare with classical RAG and Agentic RAG under shared backbone, corpus, reranker, response cap, generator, and critic. The educational instantiation shows that the same generator scores higher when fed context assembled by MARLET, with lower resource cost in several settings.

Contributions. (i) We propose MARLET, a configurable multi-agent retrieval framework with routing, specialist context construction, bounded tools, conditional retrieval, generation, and critique. (ii) We formalize transparent training-free controls over task state, answer criteria, and external evidence. (iii) We validate against classical RAG and Agentic RAG under shared infrastructure.

II. Related Work

Classical RAG fixes retrieval as the first controller: the query is matched to passages, and generation conditions on returned evidence [4]. This ordering is brittle when the same surface query should be constrained by visible state, answer criteria, or response form before evidence is selected. Recent RAG variants solve narrower control problems. Self-RAG learns reflection tokens for retrieval and critique, so it cannot be adopted without training or instruction tuning the backbone [6]. CRAG improves robustness by evaluating already-retrieved documents and optionally expanding to web search, so it corrects retrieval after the initial query rather than deciding what

Submitted to the 2026 IEEE International Conference on Systems, Man, and Cybernetics (SMC). Single-blind review.

constraints should shape that query [7]. Adaptive-RAG trains a smaller model to classify question complexity and choose among QA strategies, which controls reasoning depth but not task state, criteria, or prompt assembly [8]. TARG is training-free and efficient, but its decision is only a retrieve-or-skip gate computed from a no-context draft; it does not build typed context before retrieval [9]. RAG diagnostics further separate retrieval, ranking, and generation errors, but they diagnose failures rather than provide a controller that routes, budgets, retrieves, and assembles context [10].

Agentic RAG and multi-agent LLM systems address the flexibility missing from fixed pipelines, but their control loops are too open-ended for bounded context construction. ReAct interleaves reasoning with tool actions, while AutoGen, CAMEL, and MetaGPT coordinate conversable or role-playing agents for broad task execution [11]–[14]. MA-RAG is closer to the target setting because it decomposes ambiguous and multi-hop QA into planner, step-definition, extraction, and QA agents [15]. Its strength is iterative reasoning for difficult questions; its limitation for controlled RAG is that the agents remain part of an answer-solving chain rather than a bounded fan-out/fan-in prompt-construction contract. In a controlled RAG controller, each specialist must return a typed context block, retrieval must be skippable when unnecessary, and all intermediate outputs must converge into one final generator prompt. Without those constraints, latency and token cost are governed by the agent loop, making efficiency claims hard to audit.

Full tutoring agents are related applications, not the main comparison class. Systems such as AgentTutor assess learners, maintain memory, and adapt teaching strategies across multi-turn instruction [3]; evaluation work measures whether a produced tutor response is pedagogically useful [2], [16]–[19]. These systems may already model learner state, so criticizing them for lacking such a variable would be weak. The key mismatch is operational: they optimize an end-to-end tutoring policy with memory, interaction, and teaching strategy, whereas this paper studies the narrower RAG problem of how a fixed generator receives external evidence and task constraints. Treating full tutors as primary baselines would entangle retrieval quality with pedagogy, memory, and dialogue policy. The relevant comparison is therefore classical RAG and matched agentic RAG under the same generator, corpus, reranker, and response budget.

III. Methodology

MARLET turns one domain request into a grounded response by treating RAG as coordinated context construction. Every intermediate output becomes context for one final LLM prompt. Its design integrates three principles: route before retrieval, exchange typed records among LLM specialists, and retrieve evidence only after the response brief specifies what should be searched. The domain changes by replacing cue lexicons, default criteria, tools, and corpus; the flow remains standardize, route, run parallel specialists, execute bounded tools, retrieve conditionally, assemble one prompt, and critique once.

Algorithm 1: MARLET Inference Process

Require: entry $x = (q, o, c, h, r, v)$, caps K, Q, L, N, index $\mathcal{I}$, thresholds τ.
Return: answer y and internal grounding fields $(R, D, \text{citations})$.

$b \leftarrow \text{concat}(q, c, r, h)$; $s \leftarrow \text{CueScores}(b)$
$(R, G) \leftarrow \text{Route}(s, r, v, \tau)$
parallel do
    $p = (a, P), T_p \leftarrow \text{Planner}(x, R, G, \mathcal{U})$
    $\ell, T_\ell \leftarrow \text{State}(q, h, \mathcal{U})$
    if $r \neq \emptyset$ or $R = \text{CF}$ then $u, T_u \leftarrow \text{Criteria}(x, R, \mathcal{U})$ else $u \leftarrow$ DefaultCriteria()
end parallel
$P \leftarrow \text{cap}(P, K)$; $T \leftarrow \text{cap}(T_p \cup T_\ell \cup T_u, Q)$; $D \leftarrow \emptyset$
if $G = 1$ then
    $D \leftarrow \text{Pack}(\text{Rerank}(\text{Dedup}(\text{Search}(\mathcal{I}, P)), q), Q, L)$
end if
$y_0 \leftarrow A_{\text{gen}}(\text{Prompt}(q, o, h, p, \ell, u, T, D, c), N)$
if $D = \emptyset$ and $s_{\text{ev}} \geq \tau_2$ then
    $D \leftarrow \text{Pack}(\text{Rerank}(\text{Search}(\mathcal{I}, [q]), q), Q, L)$
    $y_0 \leftarrow A_{\text{gen}}(\text{Prompt}(q, o, h, p, \ell, u, T, D, c), N)$
end if
$y \leftarrow \text{CriticOnce}(y_0, q, h, p, \ell, u, D, r)$
return $y, (R, D, \text{ids}(D))$

A. Input Entry and Inference Caps

For one sample, MARLET first standardizes all visible fields into

$$x = (q, o, c, h, r, v). \tag{1}$$

Here, q is the question or task; o answer options; c inline context; h recent interaction history; r explicit criteria; and v images. Empty fields are allowed, so the same representation covers factual questions, assessment, feedback, and planning.

The only user-facing output is answer y. Internally, MARLET keeps a route record, evidence bundle D, tool observations T, and citations. Planner output, state summary, criteria summary, T, and D become prompt blocks. Caps keep inference bounded: K for planner/retrieval queries, Q for chunks/tool calls, L for evidence length, and N for response tokens.

The core design choice is to avoid asking one retrieval query to carry every constraint. MARLET instead fills three typed slots. The planner writes $p = (a, P)$, where a is an operating strategy and P is a capped list of searchable intents. The state diagnoser writes ℓ, a visible-state summary inferred only from q and h. The criteria analyst writes u, preserving explicit criteria r or supplying a short correctness/clarity/evidence/actionability checklist. This separates search intent, task state, and answer requirements [3], [20].

B. Supervisor Routing

The supervisor begins with a routing blob:

$$b(x) = \text{concat}(q, c, r, h), \tag{2}$$

where $\text{concat}(\cdot)$ means that the question, inline context, criteria text, and interaction turns are joined into one normalized string. Images remain available to the final generator, but they are not converted into the textual routing score.

The router uses cue families $j \in \{\mathrm{ev}, \mathrm{cr}, \mathrm{pl}, \mathrm{ad}\}$ for evidence, criteria, planning, and adaptation. These are response-construction needs, not benchmark labels. Evidence cues ask for source support; criteria cues ask for judgment against requirements; planning cues ask for ordered actions; and adaptation cues indicate state-aware delivery. Each family has keyword set W_j; let $M_j(x)$ be the number of cues from W_j in $b(x)$. The cue score is

$$s_j(x) = \frac{M_j(x)}{|W_j|}, \tag{3}$$

where $|W_j|$ is the cue-family size. Thresholds are fixed hyperparameters, not probabilities: τ_{mid} requires repeated cues, τ_{plan} is stricter because planning changes response structure, and τ_2 permits retry only for strong evidence cues. Porting mainly replaces W_j and default criteria.

The supervisor then chooses a response regime, a prompt-priority decision made before retrieval. The four regimes are planning (PL), criteria feedback (CF), adaptive response (AR), and evidence grounding (EG). PL reserves space for sequence and dependencies; CF for evaluation rules; AR for user/task state; and EG for source evidence. Because needs can overlap, $R(x)$ is selected by

$$R(x) = \begin{cases} \mathrm{PL}, & s_{\mathrm{pl}} \geq s_{\mathrm{cr}},\ s_{\mathrm{pl}} \geq s_{\mathrm{ad}},\ s_{\mathrm{pl}} \geq \tau_{\mathrm{plan}}, \\ \mathrm{CF}, & s_{\mathrm{cr}} \geq s_{\mathrm{ad}},\ s_{\mathrm{cr}} \geq \tau_{\mathrm{mid}}, \\ \mathrm{AR}, & s_{\mathrm{ad}} \geq \tau_{\mathrm{mid}}, \\ \mathrm{EG}, & \text{otherwise.} \end{cases} \tag{4}$$

The order reflects constraint strength: sequence changes response structure, criteria constrain evaluation, state shapes delivery, and evidence is the default grounding route.

Next, the supervisor uses a retrieval gate: a binary control for whether external evidence enters the final prompt. Inspired by selective-retrieval research, MARLET integrates this idea as a prompt-allocation control that opens only when outside grounding is needed [6], [8], [9]:

$$G(x) = \begin{cases} 1, & s_{\mathrm{ev}} \geq \tau_{\mathrm{mid}} \text{ or } R(x) = \mathrm{EG}, \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

Thus $G(x) = 1$ runs retrieval; $G(x) = 0$ builds the same prompt without external chunks. A stricter retry gate runs one raw-question pass only when the first bundle is empty:

$$G_2(x) = \begin{cases} 1, & D_0 \text{ is empty and } s_{\mathrm{ev}} \geq \tau_2, \\ 0, & \text{otherwise.} \end{cases} \tag{6}$$

where D_0 is the evidence bundle from the first retrieval attempt.

C. Specialist Context Agents and Coordination

After routing, MARLET knows which prompt priority must be protected: sequence, criteria, adaptation, or evidence. It then builds a response brief through LLM-backed specialist agents inspired by tool-using and multi-agent systems [11], [12]. Each specialist writes one context record rather than a final answer: it receives the standardized entry, route decision, and a bounded tool set $\mathcal{U}$; returns a JSON-like typed record; and stops. The coordinator then assembles the records into one shared context.

The three records have different downstream uses. The planner's P is the only specialist output consumed by retrieval; it converts the original request into compact search intents, which prevents long dialogue, criteria text, or user-state text from becoming retrieval noise. The state summary ℓ is used by the generator and critic to match explanation depth and response style to visible conditions. The criteria summary u is used by the generator and critic to keep the answer aligned with explicit or default requirements. When no explicit criteria are present and the route is not criteria feedback, u is filled by the same default checklist rather than by an extra LLM call.

The tools are side-effect-free local functions: key-term extraction, inline-context inspection, dialogue-state inspection, criteria listing, and corpus search. Specialists may request these tools in their JSON output, and the coordinator also provides a small default observation to each specialist. Tool observations T are capped by Q and inserted as prompt context. This gives the agents the same practical affordance used by coding assistants–inspect a local resource before writing–while keeping execution bounded.

This is also the communication pattern among agents. MARLET uses fan-out/fan-in coordination: the coordinator sends the same normalized entry to active specialists in parallel, executes validated tool calls, fills deterministic fallbacks if any output is malformed, and merges (R, G, p, ℓ, u, T) into one response brief. If $G = 1$, retrieval uses P to build evidence bundle D; if $G = 0$, the brief bypasses external chunks. In both cases, the generator receives explicit prompt blocks for plan, visible state, criteria, tool observations, and evidence instead of inferring these constraints from one overloaded query.

D. Conditional Retrieval and Evidence Assembly

After the LLM specialists produce the response brief, the coordinator uses the gate G to decide whether external evidence should enter the final prompt. If $G = 0$, the generator receives the plan, visible state, criteria, and tool observations without retrieved chunks. If $G = 1$, the planner intents P are sent to the corpus-search tool, while dialogue, criteria, and state summaries remain separate prompt blocks. This keeps retrieval focused on search intent while still preserving the other constraints for generation.

The retrieval path has four bounded steps:

$$D = \mathrm{Pack}(\mathrm{Rerank}(\mathrm{Dedup}(\mathrm{Search}(\mathcal{I}, P)), q), Q, L), \tag{7}$$

where $\mathcal{I}$ is the configured corpus index, Search retrieves candidates for the intents in P, Dedup merges repeated chunks, Rerank scores candidates against the original

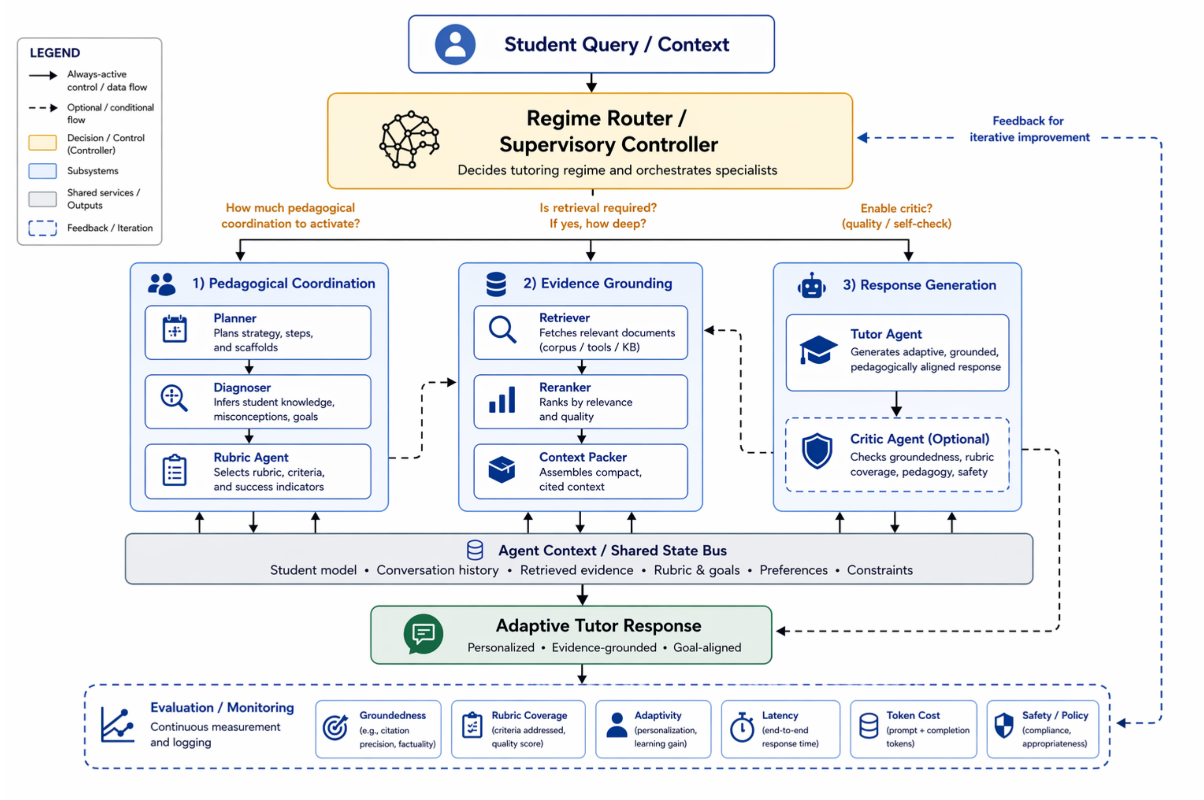

Fig. 1. Main framework of MARLET. The router scores the entry, activates specialist modules, and decides whether retrieval is needed; specialist outputs and retrieved evidence are then assembled into one augmented generator prompt.

question q, and Pack selects at most Q relevant and non-redundant snippets under length cap L. The relevance-diversity packing follows the intuition of MMR-style selection [21]. The result is the evidence bundle D, a typed prompt block that later converges with the plan, state, criteria, and tool observations.

E. Generation and Critic Revision

This is where retrieval and agent outputs converge: (R, G, p, ℓ, u, T) supplies route, plan, state, criteria, and tool-observation fields, while D supplies compact cited evidence. The final prompt contains task fields, response brief, evidence or inline context, and output requirements. The generator is the LLM-driven answer writer:

$$y_0 = A_{\text{gen}}(q, o, h, p, \ell, u, T, D, c). \tag{8}$$

Its prompt contains q, optional answer options o, recent interaction history h, state summary ℓ, plan p, criteria summary u, tool observations T, and either retrieved evidence D or inline context c. The generator is instructed by the domain configuration to be correct, useful, concise but not terse, include a next step when appropriate, and cite retrieved evidence with document markers.

The critic performs one bounded revision pass. It checks for missing citation markers when D is nonempty, uncovered criteria from r, or a response style that conflicts with the inferred state. If no issue is found, $y = y_0$; otherwise the critic rewrites once using $(y_0, q, h, p, \ell, u, D, r)$ and the revised y becomes the final answer.

IV. Validation Experiments

We validate the framework on two public educational benchmarks used as domain-specific stress tests. TutorEval [2] pairs science tutoring questions with long STEM chapters and teaching key points; EduBench [17] covers diverse educational scenarios with prompts, metadata, and reference model predictions that our adapter converts into score and rubric supervision.

The primary validation compares three context controllers under matched infrastructure, with a Qwen3.5-27B-FP8 replication checking backbone sensitivity. Classical RAG retrieves first from the raw standardized prompt; visible learner profile, student answer, dialogue, and criteria text remain available to its generator prompt, so the baseline is not deprived of task context. Its limitation is controller order: retrieval happens before specialist summaries can shape search. Agentic RAG runs the same planner, state diagnoser, and criteria module before retrieval, but retrieval is always on. MARLET keeps the same specialists and retriever, then adds supervisory routing, conditional retrieval, and retry control. We report quality and cost together because only context construction changes.

A. Implementation Details

All controllers use the same deployed Python framework, corpus, hybrid text index, reranker, generator, critic, cache policy, and metric code; only context assembly differs. The planner, state diagnoser, criteria module, generator, and critic use the same configured backbone; deterministic fallbacks run only when an LLM response is unavailable or malformed. MARLET enables bounded local tool calls for specialist observations and corpus search, with no network, file mutation, or open-ended shell access. LLM endpoints are served through SGLang, whose runtime is designed for structured multi-call LLM programs and KV-cache reuse [22]. We also enable exact model-response caching for every controller because repeated prompts, fixed criteria, and repeated served requests are normal in deployed systems; a hit requires an identical full payload and is marked in the result artifacts. Therefore latency is cache-aware served-system wall time, not a cold-start model benchmark. Runs use Qwen3.5-4B and Qwen3.5-27B-FP8, reasoning budget 512, generation temperature 0.15, specialist JSON temperature 0.0, and response cap N=900 on benwulab-Lambda-Vector with two NVIDIA RTX 6000 Ada GPUs. We set K=3, Q=4, L=6000, and router thresholds $(\tau_{\text{mid}}, \tau_{\text{plan}}, \tau_2) = (0.35, 0.42, 0.45)$.

Evaluation is dataset-aware, and all quality scores are automatic proxies rather than official benchmark or human grades. Inspired by RAG evaluation work [10], we separate answer overlap, criteria fulfillment, grounding, and resource cost. Token-F1 balances reference-token coverage against extra generated tokens; rubric coverage checks whether benchmark criteria are expressed with exact or content-bearing partial matches; latency is served wall-clock seconds per sample under the shared cache policy; and tokens count prompt plus completion tokens.

For EduBench, EB12D averages 12 normalized checks over scenario adaptation, factual/reasoning behavior, and pedagogical application, while EduAlign only measures extracted-score agreement with the benchmark consensus. For TutorEval, tutor-hit measures expert teaching-point coverage with full, partial, or zero credit.

B. Framework Validation

Table I reports paired 4B comparisons with bootstrap CIs and permutation p-values; Table II reports the 27B replication. Agentic RAG uses the same specialist brief as MARLET but retrieves for every sample, and its matched results are being generated under the same protocol.

C. Trade-offs

For TutorEval, the 27B paired CIs are positive for Token-F1, tutor-hit, rubric coverage, and latency. For EduBench, EB12D and rubric coverage improve, latency and token use decrease, and the small classical-RAG EduAlign edge is not statistically reliable.

MARLET can use more modules than classical RAG, so resource cost must be reported. Scoped planner queries and the retrieval gate reduce token use in 4B and still reduce latency in 27B, but 27B TutorEval shows that better coverage does not always mean fewer tokens.

TABLE I

4B quality and resource summary.

Data	Metric	RAG	Ours	Δ
TutorEval	Token-F1	0.112	0.115	$+0.004^{\star}$
	TE hit	0.938	0.957	$+0.019^{\star\star}$
	Rubric	0.919	0.944	$+0.026^{\star\star}$
	Latency (s)	51.90	49.11	-2.79
	Tokens	3975	3490	$-484^{\star\star\star}$
EduBench	EB12D	0.367	0.398	$+0.031^{\star\star\star}$
	Rubric	0.705	0.891	$+0.186^{\star\star\star}$
	EduAlign	0.166	0.126	$-0.040^{\star\star}$
	Latency (s)	19.83	13.90	$-5.92^{\star\star\star}$
	Tokens	4020	2226	$-1794^{\star\star\star}$

$^{\star\star\star}p<10^{-4}$, $^{\star\star}p<0.01$, $^{\star}p<0.05$. Higher is better for quality; lower is better for latency and tokens. Runtime is mean wall-clock seconds/sample on the experiment server.

TABLE II

27B quality and resource summary.

Data	Metric	RAG	Ours	Δ
TutorEval	Token-F1	0.264	0.272	$+0.008^{\star}$
	TE hit	0.897	0.918	$+0.021^{\star}$
	Rubric	0.861	0.891	$+0.030^{\star}$
	Latency (s)	41.47	40.65	$-0.82^{\star\star}$
	Tokens	6681	6730	$+49$
EduBench	EB12D	0.355	0.425	$+0.070^{\star\star\star}$
	Rubric	0.634	0.769	$+0.135^{\star\star\star}$
	EduAlign	0.541	0.526	-0.015
	Latency (s)	41.91	32.09	$-9.81^{\star\star\star}$
	Tokens	6105	3717	$-2388^{\star\star\star}$

TutorEval uses n=400 paired samples; EduBench uses n=328 unique paired rows after duplicate public IDs in the 400-example slice are collapsed. $^{\star\star\star}p<10^{-4}$, $^{\star\star}p<0.01$, $^{\star}p<0.05$. Higher is better for quality; lower is better for latency and tokens. Runtime is mean wall-clock seconds/sample on the experiment server.

Thus MARLET decides what grounding a request needs; it is not a universal retrieval replacement.

V. Limitations

The study validates a framework rather than a deployable assistant: evaluation is automatic and English-only, and user or task state is inferred from visible prompt/dialogue fields. Deployment requires privacy safeguards, bias checks, data governance, human oversight, and domain-specific outcome studies.

VI. Conclusion

MARLET reframes grounding as supervised context construction rather than query-only passage lookup. With shared infrastructure, the educational instantiation improves fixed-generator quality and efficiency over classical RAG by coordinating task state, answer criteria, and conditional evidence before generation. Future work should test stronger controller baselines, additional domains, human and multi-turn outcomes, learned router calibration, profile-aware reranking, and privacy-preserving state inference.

References

[1] E. Dinan, S. Roller, K. Shuster, A. Fan, M. Auli, and J. Weston, “Wizard of Wikipedia: Knowledge-powered conversational agents,” in Proceedings of the International Conference on Learning Representations, 2019. [Online]. Available: https://arxiv.org/abs/1811.01241

[2] A. Chevalier, J. Geng, A. Wettig, H. Chen, S. Mizera, T. Annala, M. J. Aragon, A. R. Fanlo, S. Frieder, S. Machado, A. Prabhakar, E. Thieu, J. T. Wang, Z. Wang, X. Wu, M. Xia, W. Xia, J. Yu, J.-J. Zhu, Z. J. Ren, S. Arora, and D. Chen, "Language models as science tutors," arXiv:2402.11111, 2024. [Online]. Available: https://arxiv.org/abs/2402.11111

[3] Y. Liu, Z. Song, J. Lou, C. Wu, and J. Li, "AgentTutor: Empowering personalized learning with multi-turn interactive teaching in intelligent education systems," arXiv:2601.04219, 2026. [Online]. Available: https://arxiv.org/abs/2601.04219

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[5] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2021. [Online]. Available: https://openreview.net/forum?id=wCu6T5xFjeJ

[6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in International Conference on Learning Representations, 2024. [Online]. Available: https://arxiv.org/abs/2310.11511

[7] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," arXiv:2401.15884, 2024. [Online]. Available: https://arxiv.org/abs/2401.15884

[8] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 7036–7050. [Online]. Available: https://aclanthology.org/2024.naacl-long.389/

[9] Y. Wang, L. Wei, and H. Ling, "Retrieval as a decision: Training-free adaptive gating for efficient RAG," arXiv:2511.09803, 2026. [Online]. Available: https://arxiv.org/abs/2511.09803

[10] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, C. Jiayang, C. Wang, S. Sun, H. Li, Z. Zhang, B. Wang, J. Jiang, T. He, Z. Wang, P. Liu, Y. Zhang, and Z. Zhang, "RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation," 2024. [Online]. Available: https://arxiv.org/abs/2408.08067

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in International Conference on Learning Representations, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[12] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2024. [Online]. Available: https://arxiv.org/abs/2308.08155

[13] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large language model society," 2023. [Online]. Available: https://arxiv.org/abs/2303.17760

[14] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," 2024. [Online]. Available: https://arxiv.org/abs/2308.00352

[15] T. Nguyen, P. Chin, and Y.-W. Tai, "MA-RAG: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning," arXiv:2505.20096, 2025. [Online]. Available: https://arxiv.org/abs/2505.20096

[16] K. K. Maurya, K. A. Srivatsa, K. Petukhova, and E. Kochmar, "Unifying AI tutor evaluation: An evaluation taxonomy for pedagogical ability assessment of LLM-powered AI tutors," in Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), 2025. [Online]. Available: https://aclanthology.org/2025.naacl-long.57/

[17] B. Xu, Y. Bai, H. Sun, Y. Lin, S. Liu, X. Liang, Y. Li, Z. Dong, J. Zhang, Y. Deng, X. Zou, Y. Gao, and H. Huang, "EduBench: A comprehensive benchmarking dataset for evaluating large language models in diverse educational scenarios," arXiv:2505.16160, 2025. [Online]. Available: https://arxiv.org/abs/2505.16160

[18] J. Macina, N. Daheim, I. Hakimi, M. Kapur, I. Gurevych, and M. Sachan, "MathTutorBench: A benchmark for measuring open-ended pedagogical capabilities of LLM tutors," in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 204–221. [Online]. Available: https://aclanthology.org/2025.emnlp-main.11/

[19] R. S. Srinivasa, Z. Che, C. B. C. Zhang, D. Mares, E. Hernandez, J. Park, D. Lee, G. Mangialardi, C. Ng, E.-Y. H. Cardona, A. Gunjal, Y. He, B. Liu, and C. Xing, "TutorBench: A benchmark to assess tutoring capabilities of large language models," arXiv:2510.02663, 2025. [Online]. Available: https://arxiv.org/abs/2510.02663

[20] C. Borchers and T. Shou, "Can large language models match tutoring system adaptivity? A benchmarking study," in Artificial Intelligence in Education. Springer Nature Switzerland, 2025, pp. 407–420. [Online]. Available: https://arxiv.org/abs/2504.05570

[21] J. Carbonell and J. Goldstein, "The use of mmr, diversity-based reranking for reordering documents and producing summaries," in Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, 1998, pp. 335–336.

[22] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in Advances in Neural Information Processing Systems, vol. 37, 2024, pp. 62 557–62 583. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html